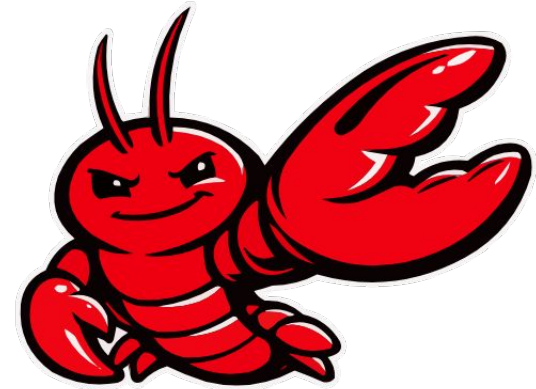


// a talk

building personal assistants on AWS w/ OpenClaw.



Lessons & takeaways from launching a 24/7 personal assistant for \$40/mo

hi, I'm ford!

- sr. staff swe & tech evangelist @ GEICO
- also: chief hacker @ hack for rva
- Richmond native
- Father of 3



> what conditions led us here?

the right tool for the right time

changed:

- ops inference \$
- dev inference \$

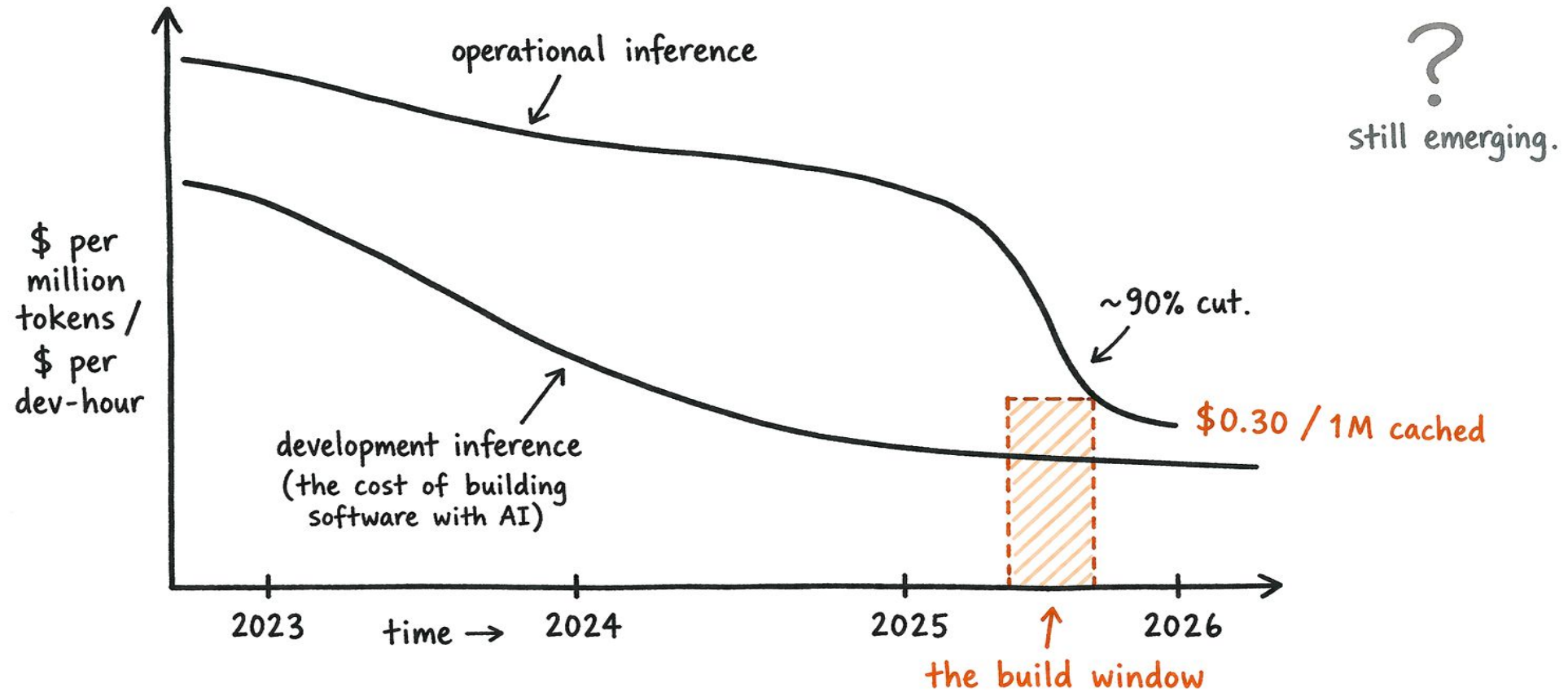
unchanged:

- integrations
- platform
- dependencies
- intelligence
- demand
- arguably: tool-calling

tl;dr - no breakthrough - a convergence of cost, capability, and timing.

> what conditions led us here?

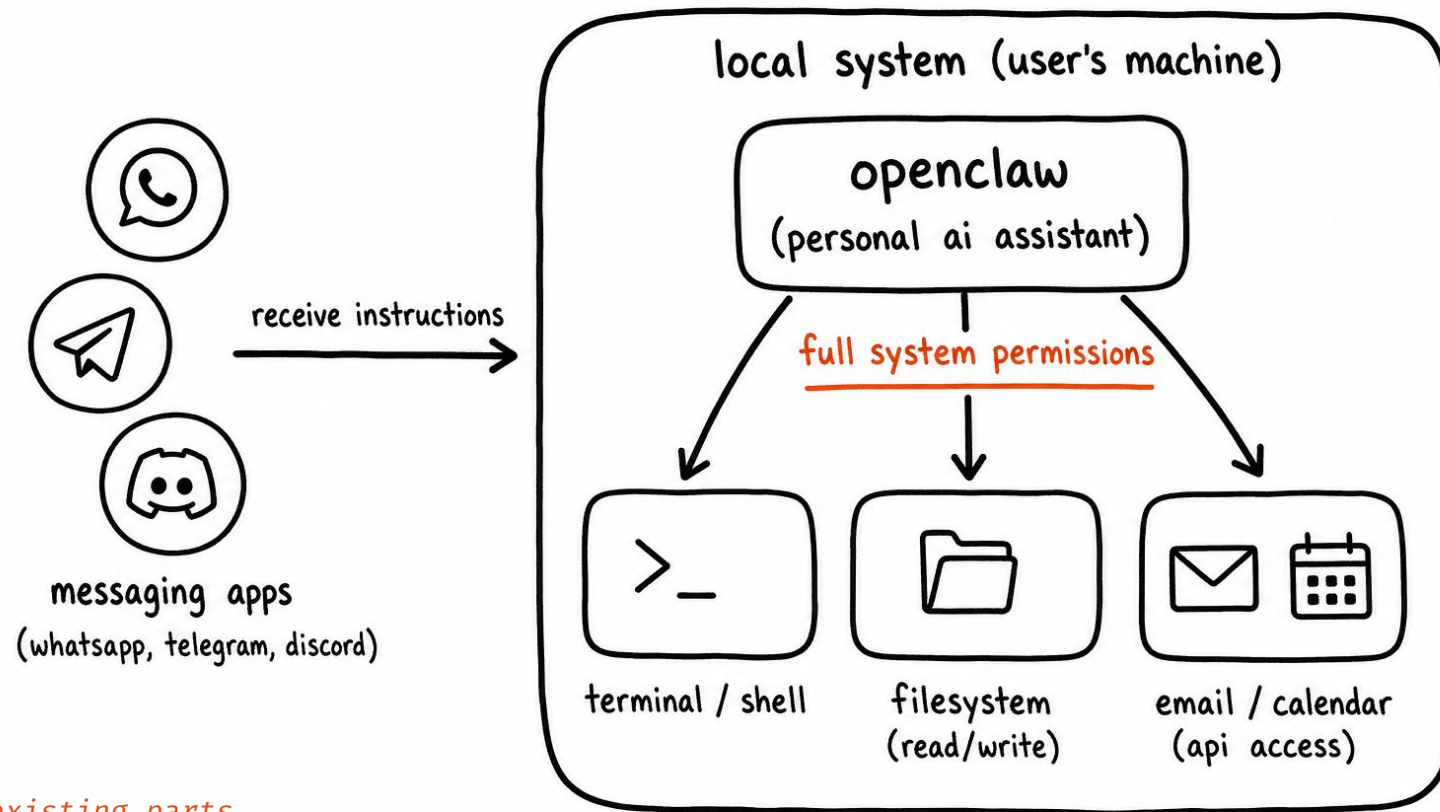
finally in scope



tl;dr - no breakthrough - a convergence of cost, capability, and timing.

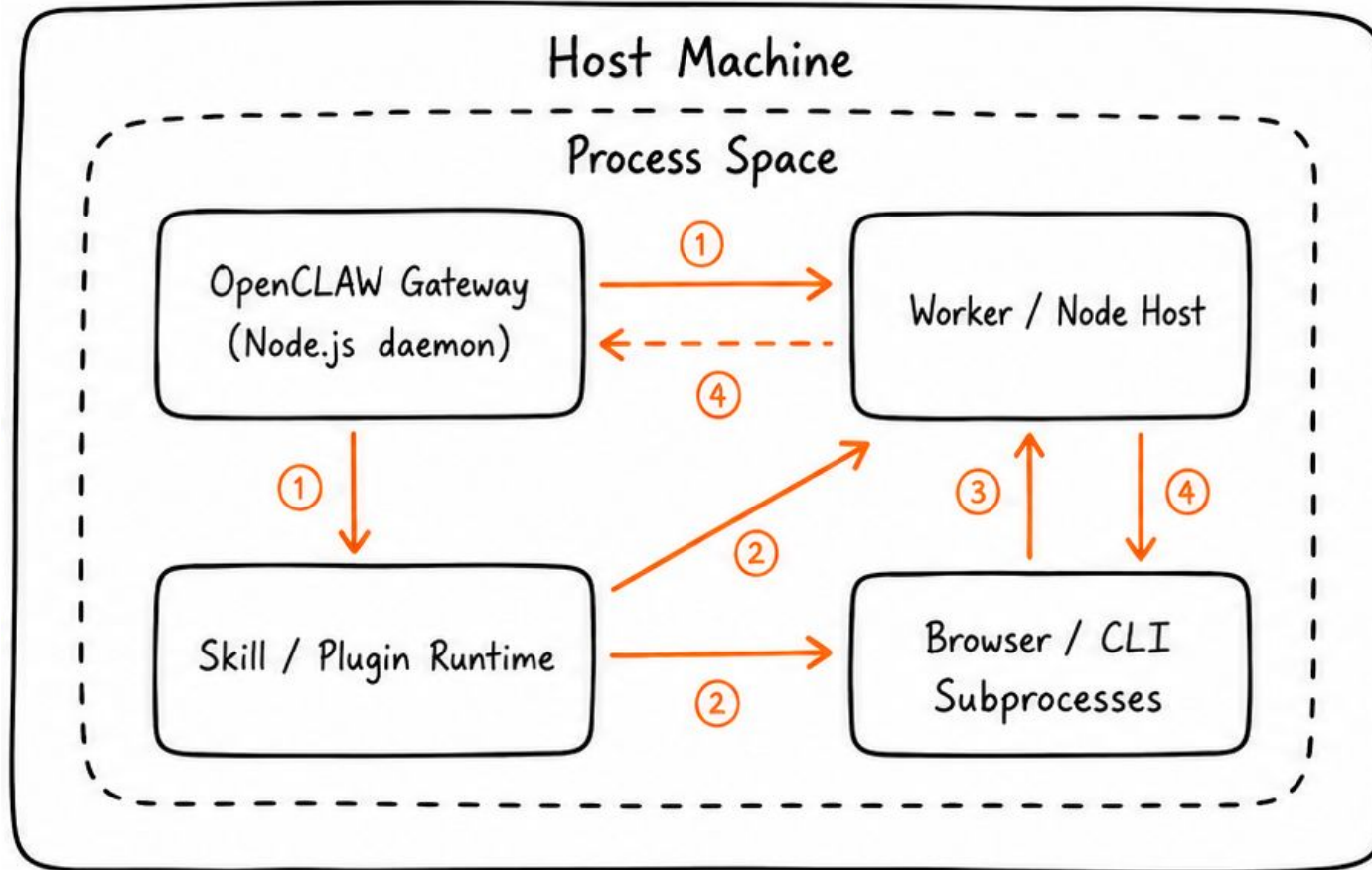
> the substrate

openclaw is a persistent agent that talks via messaging apps

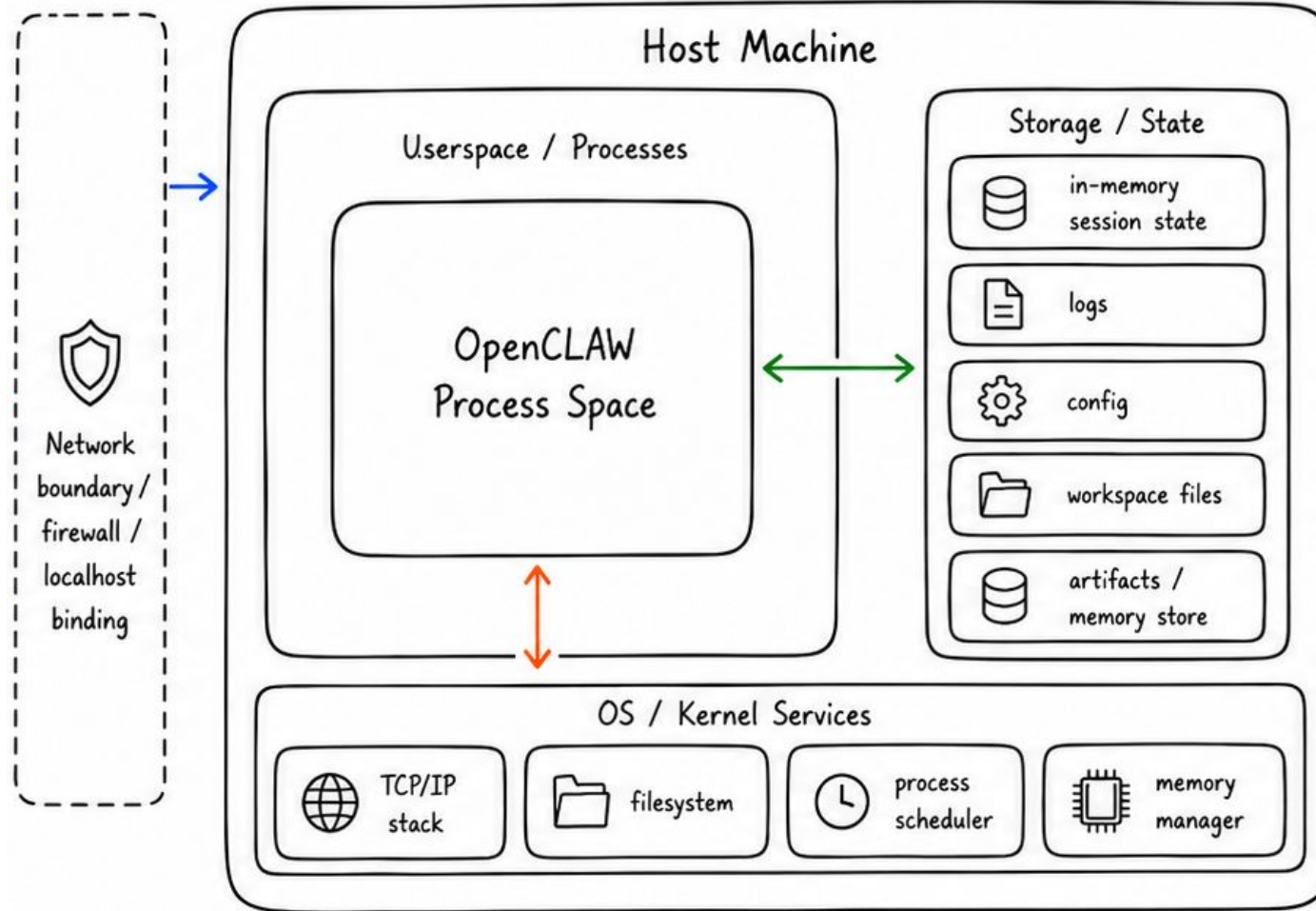


it's a composition of existing parts.

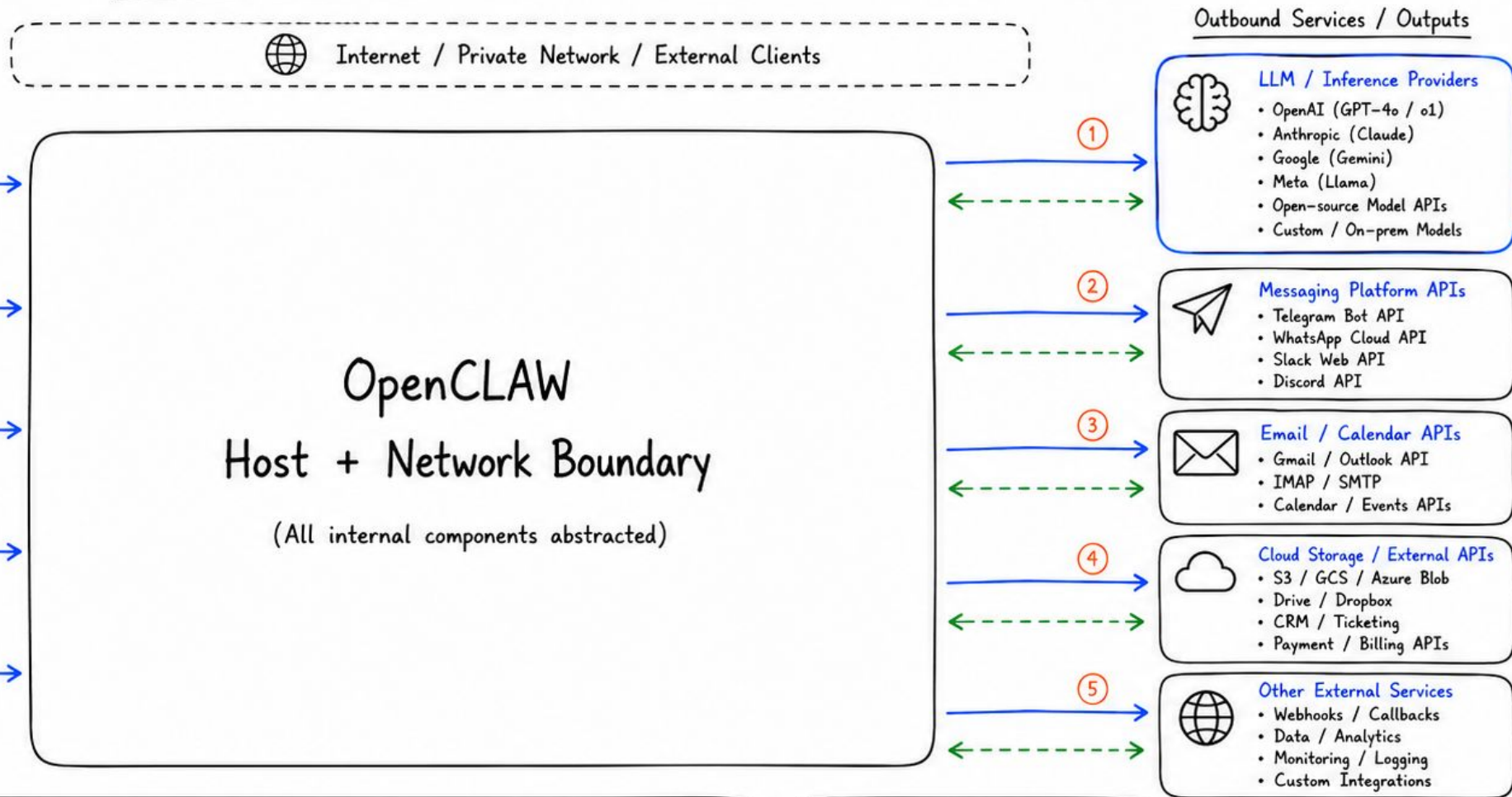
OpenCLAW: Process Space View



OpenCLAW: Host + Network Boundary



OpenCLAW: Full Network + Service Ecosystem View



End-to-End Flow Summary (High Level)

- 1 Input arrives from user / system / schedule / webhook.
- 2 Request enters OpenCLAW across the network boundary.
- 3 OpenCLAW processes the request and orchestrates work.

- 4 OpenCLAW calls external services (inference, APIs, storage, etc.).
- 5 External services respond with results / data.
- 6 OpenCLAW formats the response and returns it.
- 7 Response is delivered back to the originating source.

Legend

- Network Traffic (Ingress / Egress) (HTTPS / WSS / API Calls)
- Process / Control Flow (Tasks, Plans, Decisions)
- State / Artifact Flow (Results, Data, Files)

Notes

- 🔒 All traffic encrypted (TLS 1.2+).
- 🔑 Least-privilege access & rate limits.
- 📋 External calls follow allowlists and egress policies.
- 📊 Observability via logs, metrics, and traces.

OPENCLAW INNER CORE: MEMORY + TOOLS MAKE THE MODEL USEFUL

USER SURFACE

messages come in from anywhere

WhatsApp Telegram Discord Slack Teams iMessage ... etc.

LEGEND

- control / data flow
- user message flow
- memory write
- memory read (search)

GATEWAY (one per machine)

WebSocket
control plane
port 18789
localhost only
by default

- session routing
- connection management
- queues & concurrency
- dashboard / RPC
- auth & permissions



single long-running process
manages everything

TOOLS (outside the model)



Web_fetch
HTTP GET + extract
readable content
(no JS execution)



browser
automate web UI
(Playwright)



shell
run commands
on the machine



files
read, write, edit,
search files



APIs
call external
services



custom tools
anything via skills
or builtins

tools are sandboxed
& permissioned

AGENT LOOP (runs continuously)

- 1 load context + memory
- 2 call model (Claude / GPT / DeepSeek / etc.)
- 3 model decides next action (tool or reply)
- 4 tool runs outside the model
- 5 observation / result returns
- 6 model reasons with new info
- 7 reply streams back to user
- 8 transcript + memory persist



write
new memories

3 FILE WATCHER
watches memory files
marks index dirty
on changes
debounced reindex
keeps index fresh

read / search
relevant memories

MEMORY CORE

1 SOURCE OF TRUTH (on disk)

MEMORY.md
- facts
- preferences
- decisions

memory/YYYY-MM-DD.md
- daily notes
- events
- context

plain
Markdown
human readable
versionable

2 SEARCH INDEX (per-agent SQLite DB)

SQLite

FTSS / BM25
keyword search
exact phrases,
names, ports,
commands, projects

EMBEDDINGS (optional)
vector search
semantic similarity
when provider
is configured

4 HYBRID RECALL

merge keyword + semantic results
rerank (recency boost, diversity)
return top relevant chunks

KEY PRINCIPLE

the model supplies judgment.
the gateway supplies continuity,
tools, memory, and side effects.

SKILLS (extensible capabilities)



GitHub
integration



Smart Home
control



Workflow
automation



Database
access



...
and thousands
more

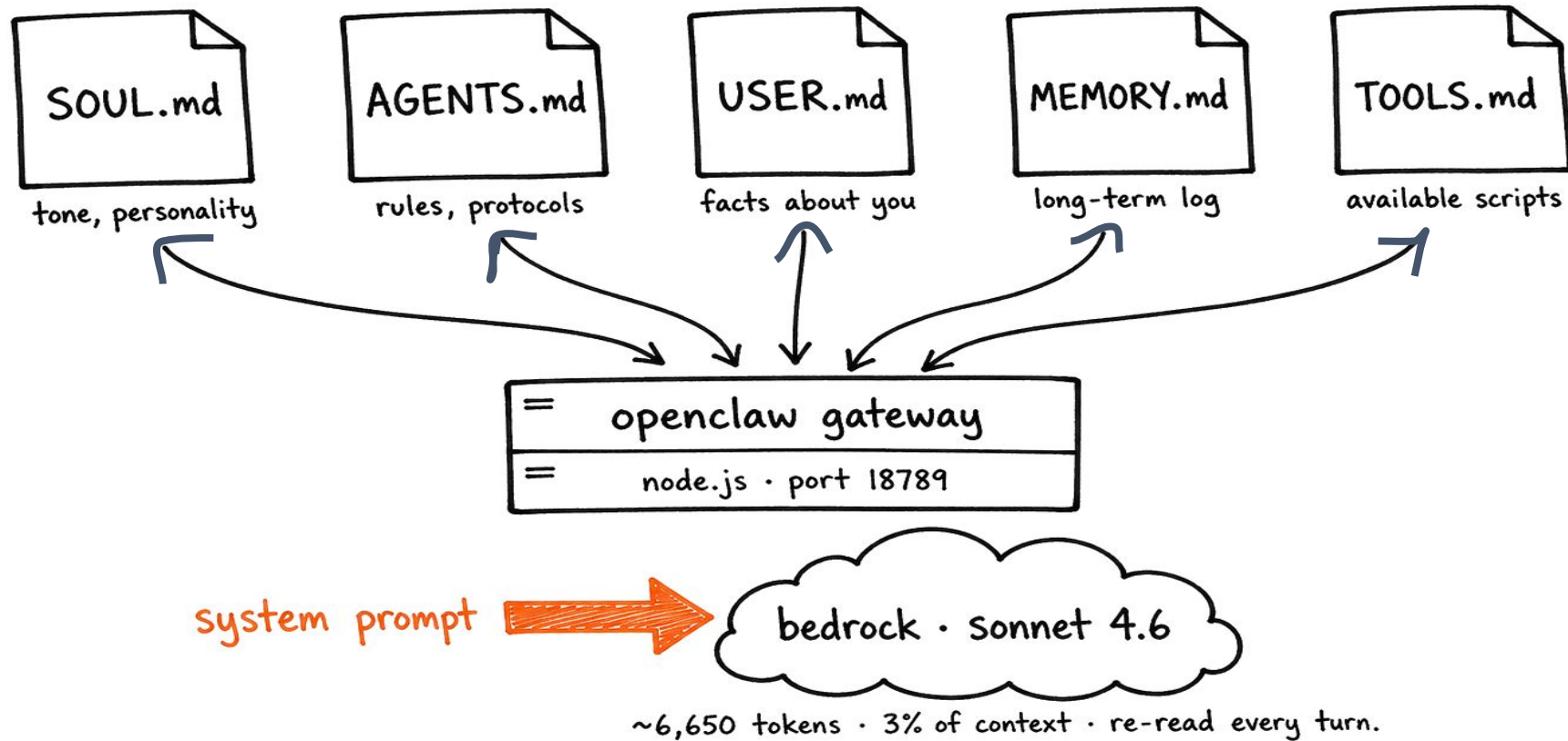
publish to CLAWHUB (public registry) → 5,000+ packages within weeks

THE RESULT

- ✓ agent remembers across days
- ✓ picks up where it left off
- ✓ combines exact recall + semantic recall
- ✓ uses tools to act in the real world
- ✓ saves everything to files you own

> the substrate

deploy target = filesystem on EC2



it's a composition of existing parts.

> actors, pipelines, ecosystems

3 new problem spaces unlocked

01 • ACT

tool switching • failure recovery •
alternative strategy.

*"couldn't book on OpenTable,
so it called the restaurant."*

the agent stopped being a
chatbot and became a
decision-making actor.

02 • PERSIST

ongoing ingestion • transformation •
scheduled synthesis.

*"ingests newsletters daily,
rewrites them into a
personalized digest."*

a living data pipeline
with opinions.

03 • INTERACT

shared space • agent-to-agent
content • emergent structure.

*"agents on Moltbook
started a religion."*

the Church of Molt. coordination
is coming. less lobster-themed.

this is how new paradigms show up – messy, social, chaotic.

> unprecedented adoption + momentum

#1 on GitHub

(surpassed
React in 100
days)



if you remember when 'cloud-native' became a thing, this is that moment for agents

> how do you keep an agent alive?

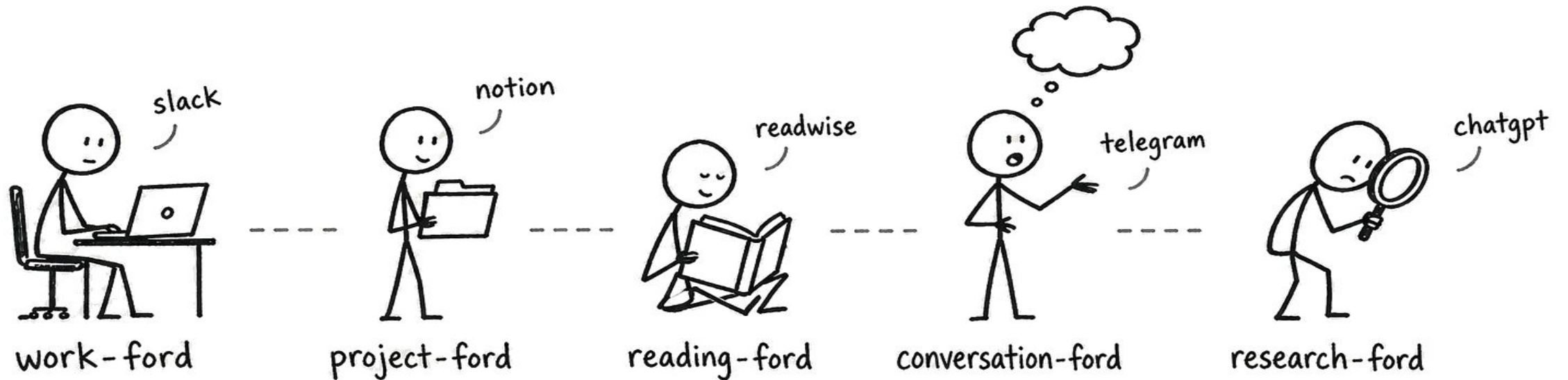
openclaw on aws?

<u>service</u>	<u>architect take</u>
EC2 / Graviton	safest baseline ✓ c7g.large default • SSM • local state
Lightsail	workshop simplest ✓ \$10 IPv6 min • \$20 comfy
Fargate + EFS	works, but slow persistent shared state Git metadata tax up to 50x slower
Bedrock AgentCore	promising active compute only model cost still matters
Lambda	conditional 15-min cap • endpoints avoid NAT \$\$
App Runner	skip closed to new customers • use ECS Express

you're not choosing services – you're choosing how state lives.

> what i actually wanted

how might I turn this...



into **one thread**, connecting all the parts of my life.

from broken productivity tool(s) to unified thinking system.

> current state

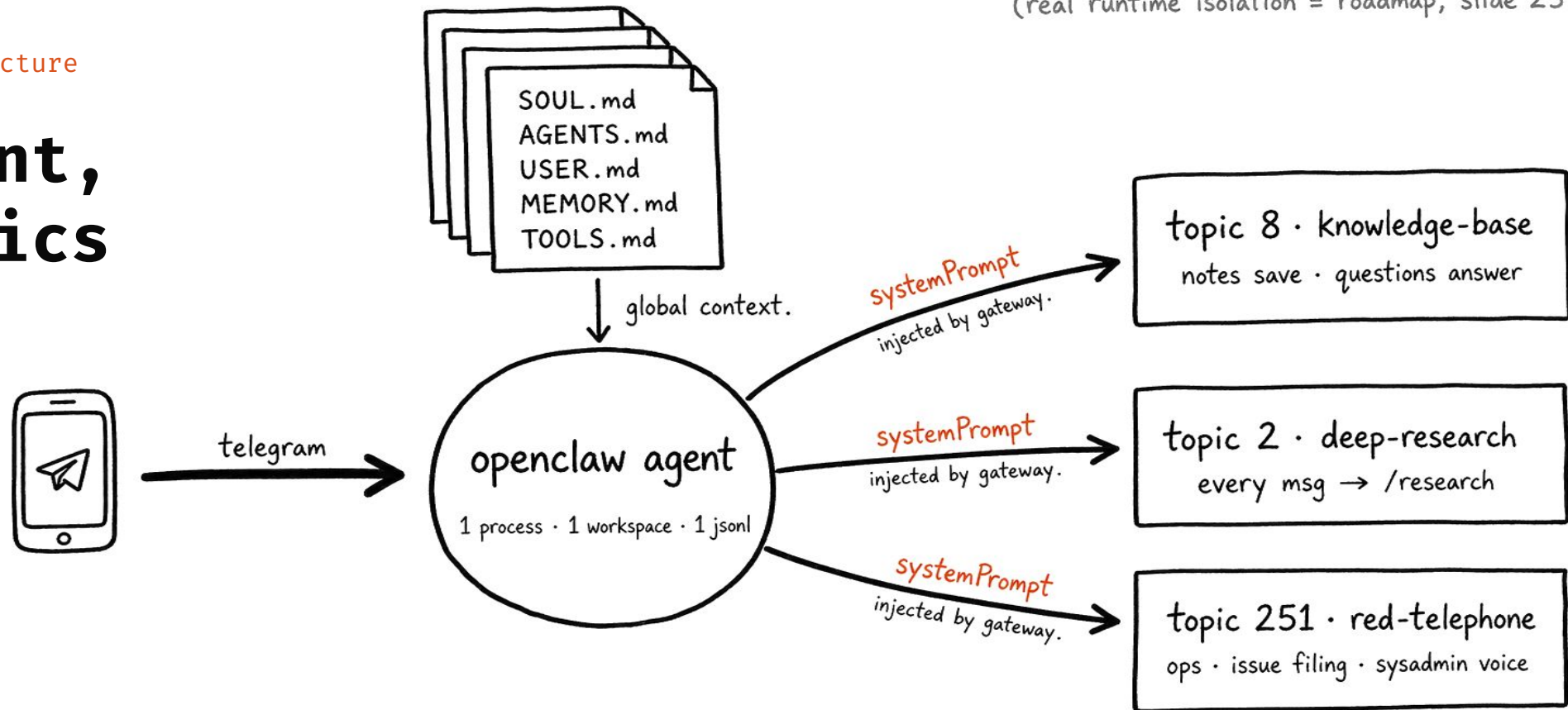
core jobs to be done:

- 01 • capture → **organize** (voice-first)
- 02 • query → **understand** (context-aware)
- 03 • research → **integrate** (structured)

not storing information – maintaining clarity across contexts.

> the architecture

1 agent, 3 topics



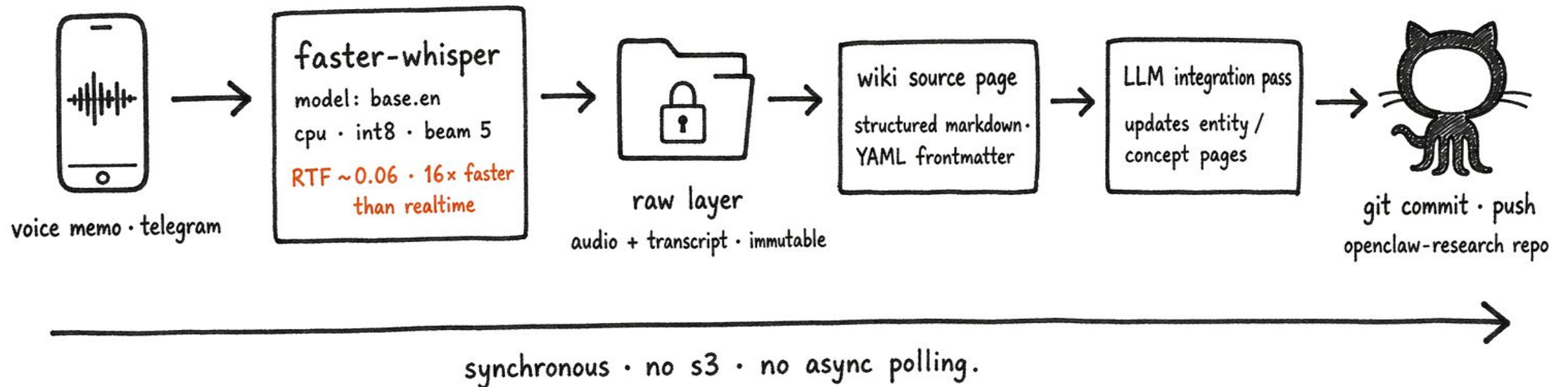
per-topic prompts steer behavior. shared workspace. shared transcript.

separation by skill, not by runtime.

separation by skill, not by runtime.

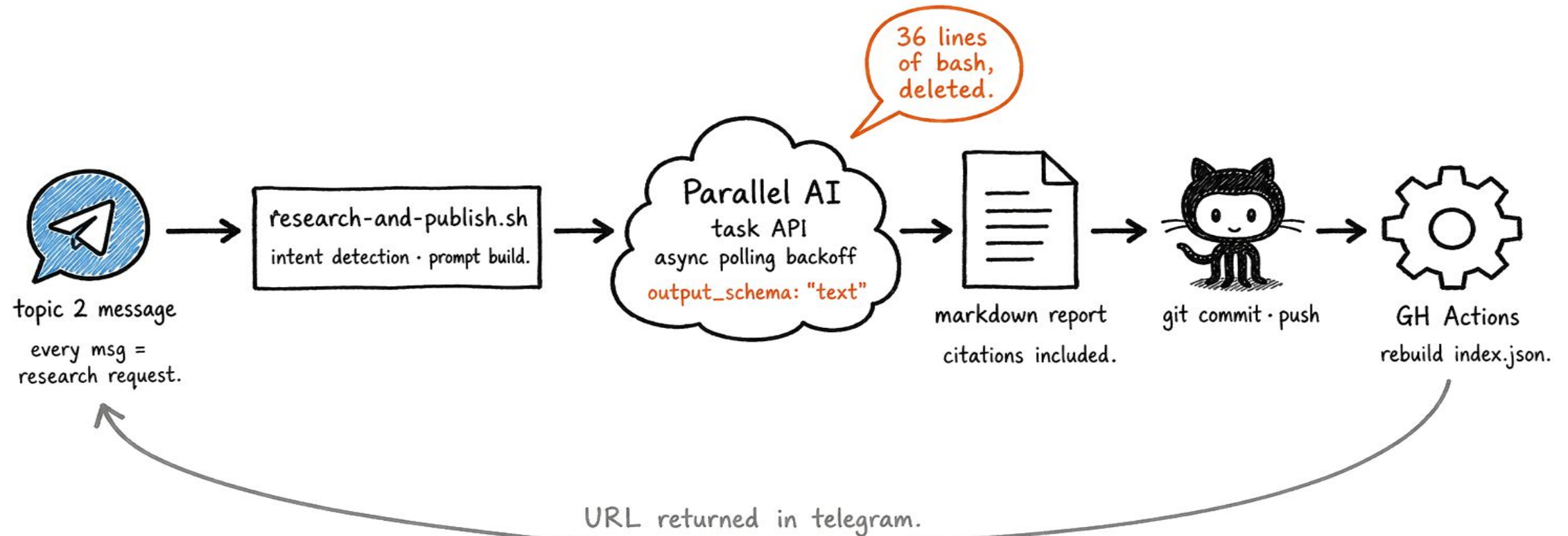
01 • knowledge-base

memory is ~250 MB.
fits a 4 GB box.
large-v3 would OOM.



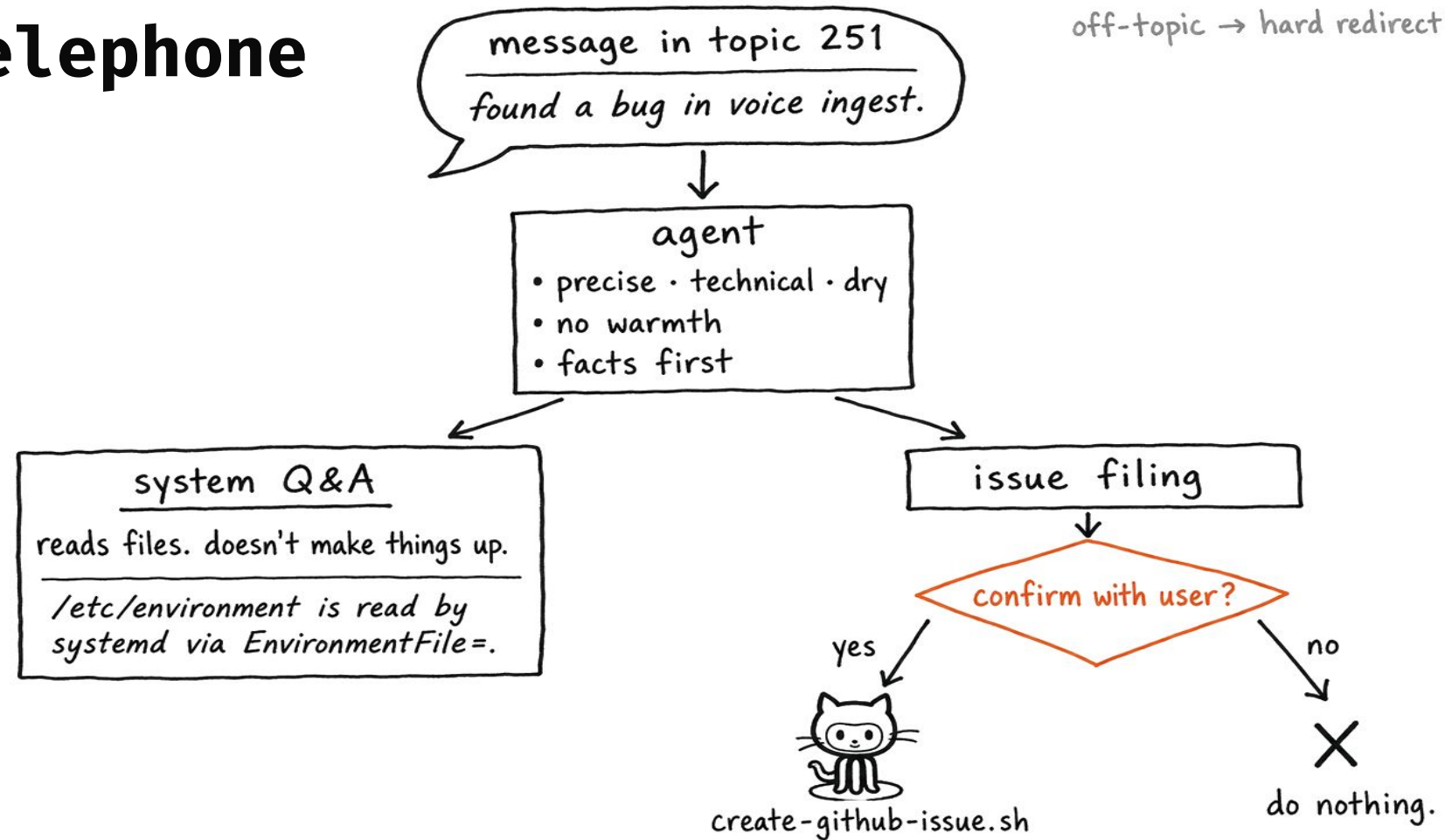
capture is automatic. organization is inferred.

02 • deep-research



from idea → investigation → knowledge, in one loop.

03 • red-telephone



every always-on agent needs an operator line.

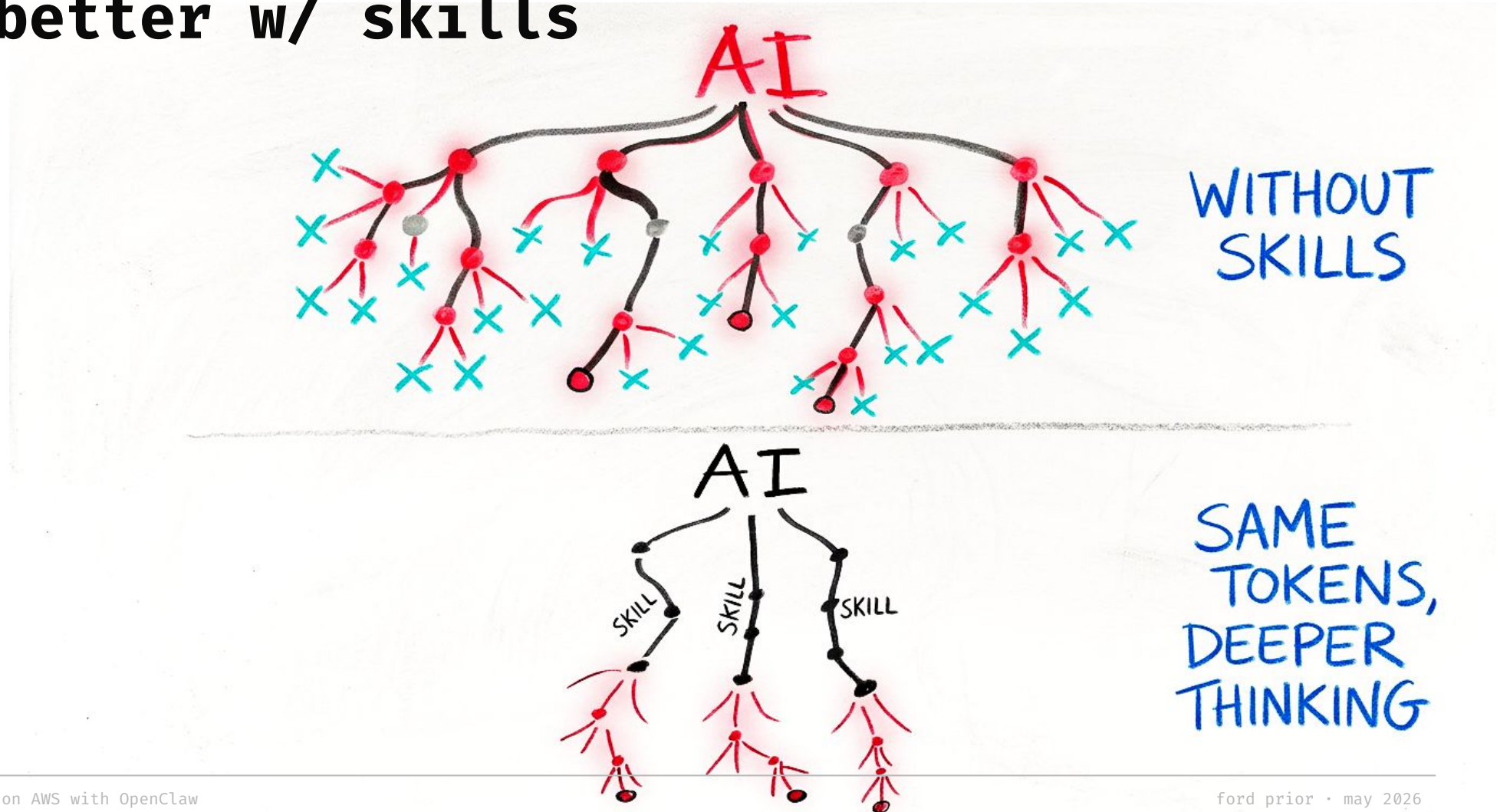
> // live demo

live demo

- send a voice memo. watch the wiki commit.

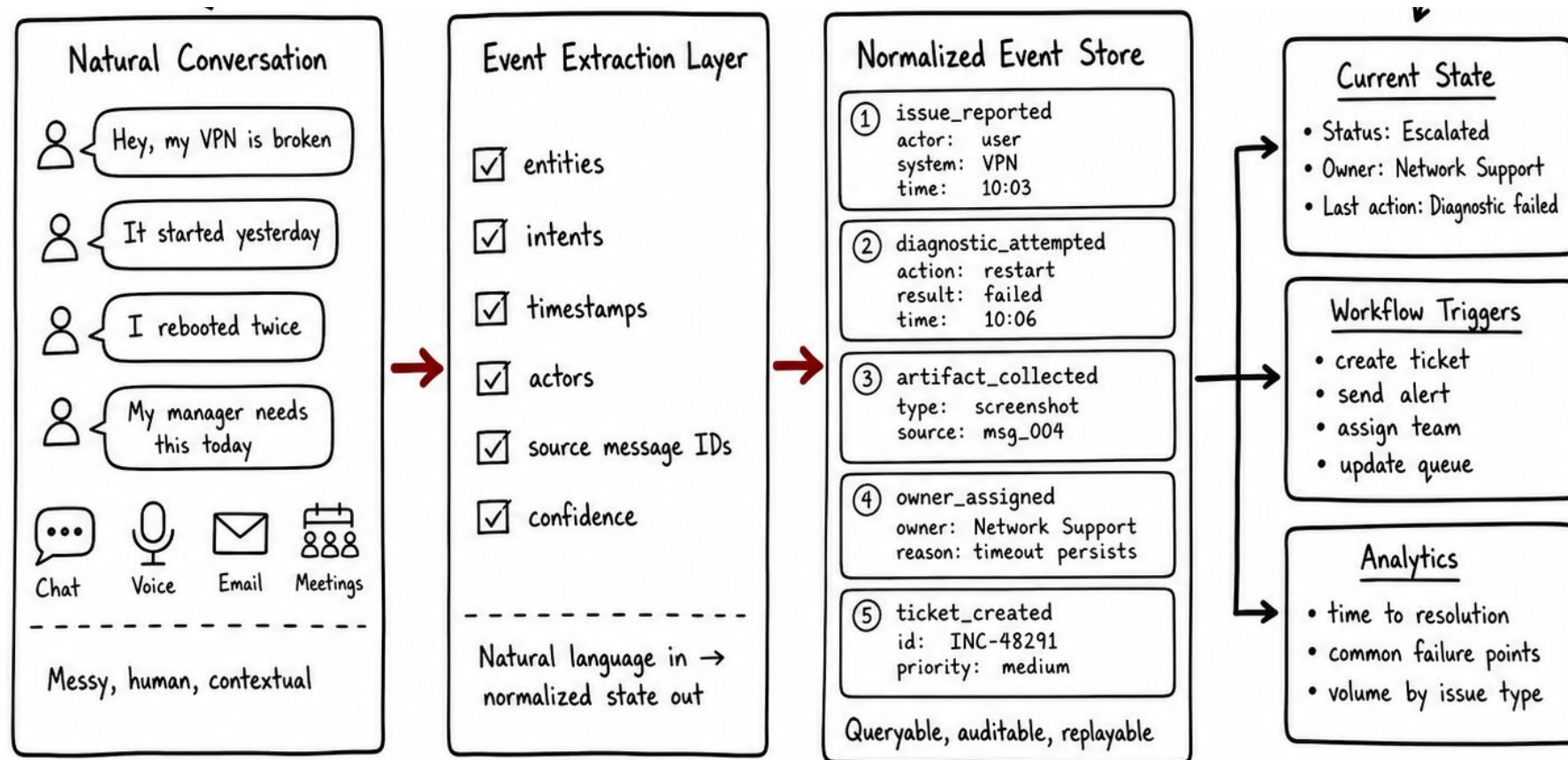
> // lesson #1

life is better w/ skills



> // lesson #2

also: events



> // lesson #3

“I’m the problem; it’s me”



- calendar out of date
- email disorganized
- unable to communicate coherently
 - asking questions
 - explaining situations

retro > v1



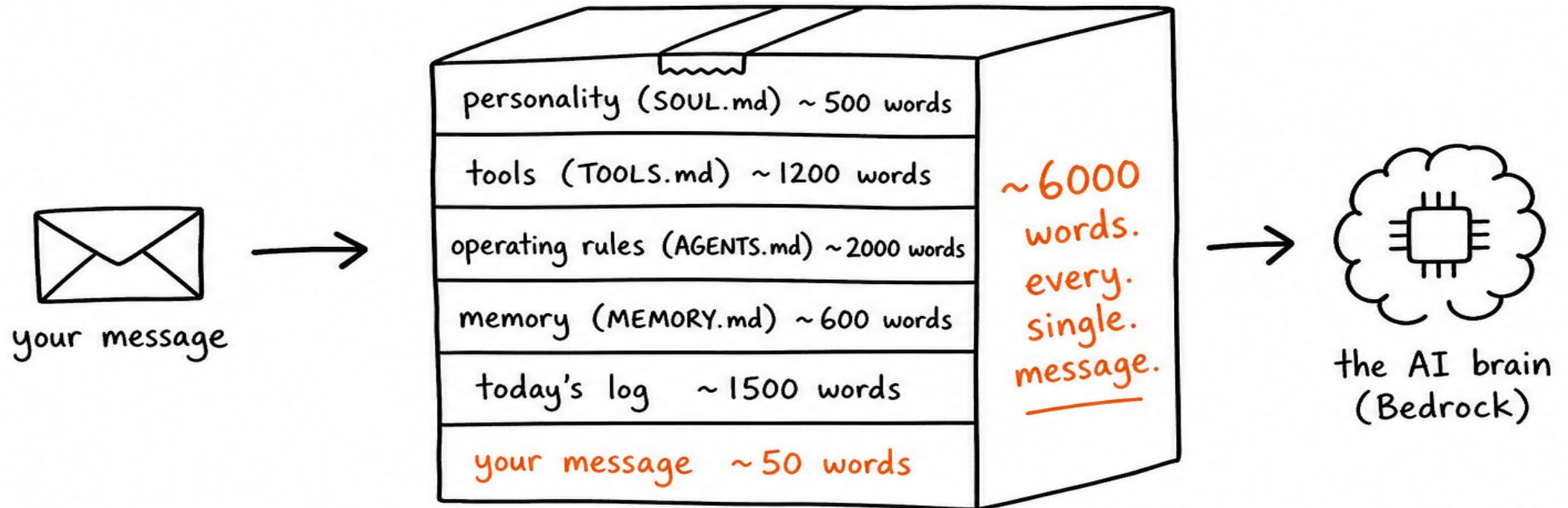
03 interesting bugs

TTL has to match human rhythm, not system defaults.

> why caching exists in the first place



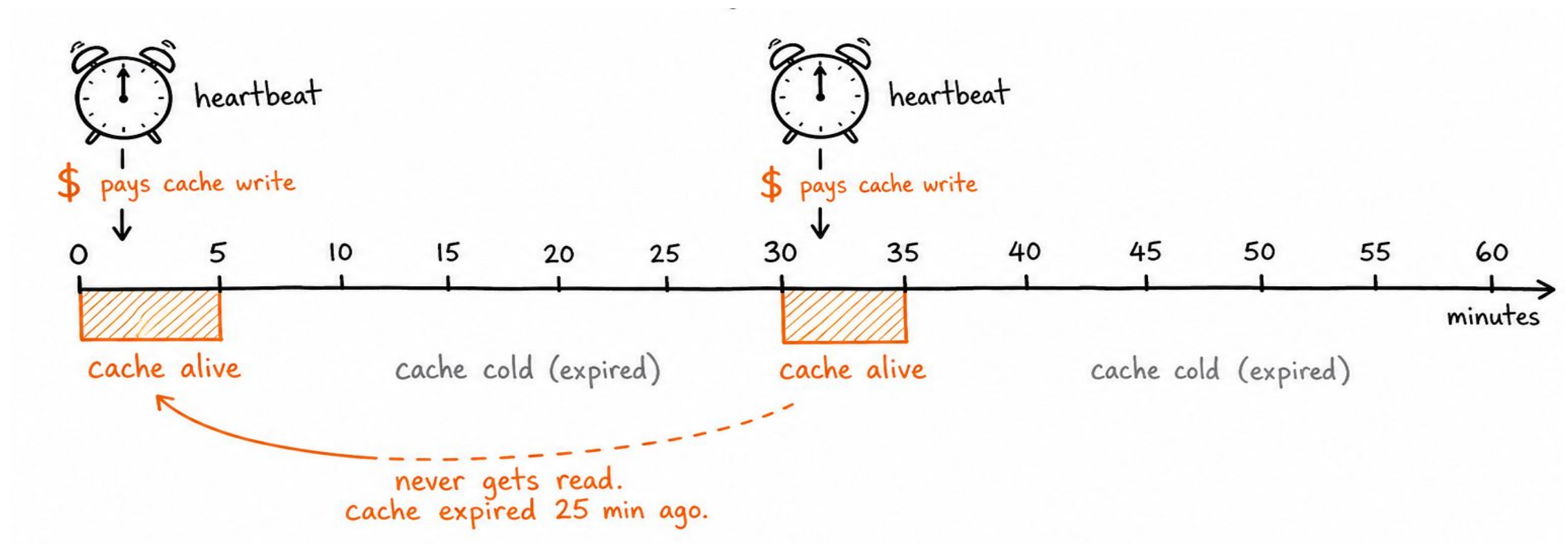
01 cache-write tax



heartbeats cost ~6k tokens

> the heartbeat fires every 30m. the cache expires every 5.

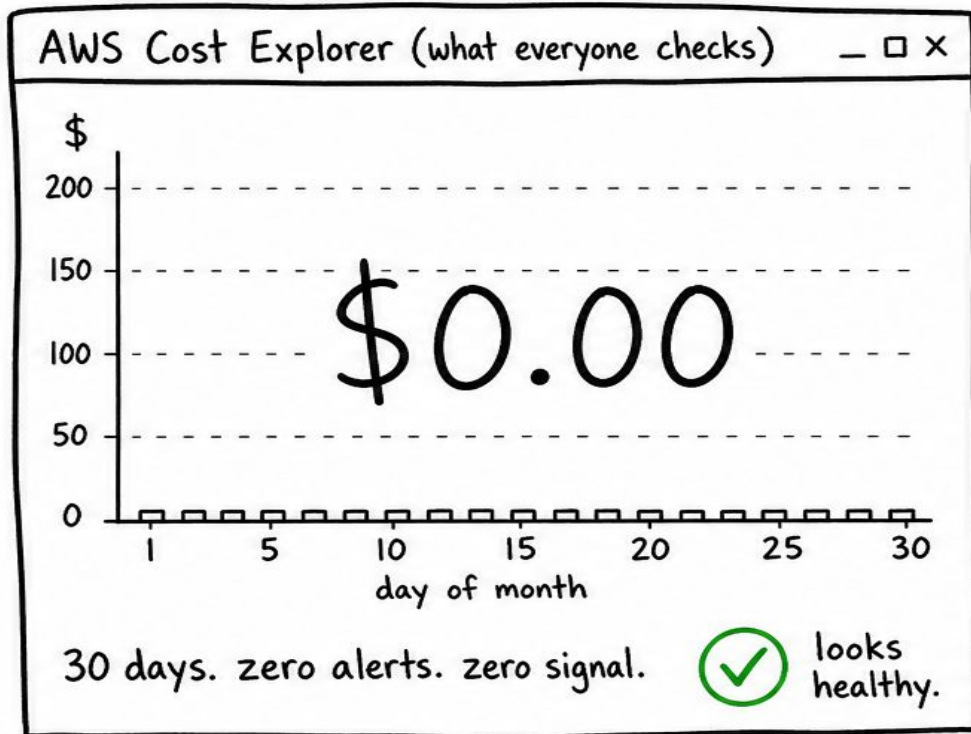
(cache) econ 101 • simple math overlooked



two reasonable defaults composing into a third behavior nobody designed.

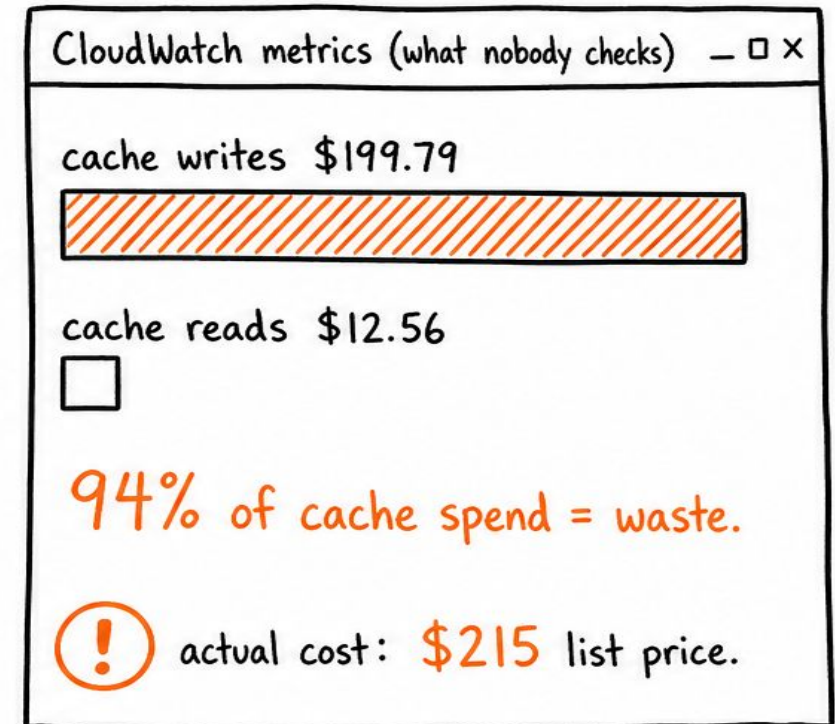
> credits hide bugs until they don't

(cache) econ 101 • the monitor lied



?

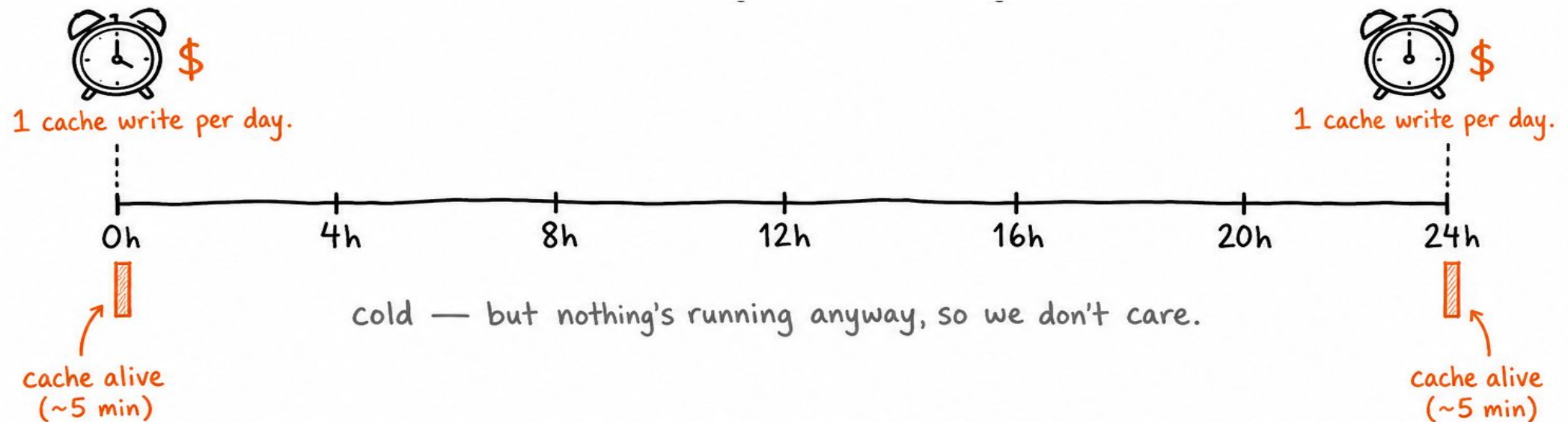
same month.
same account.
wildly different
stories.



when the bill is hidden, the CloudWatch metrics were the only signal.

> the fix was four lines of JSON

(cache) econ 101 • fixes are simple



before: 48 wake-ups/day • ~\$7/day • ~\$215/month

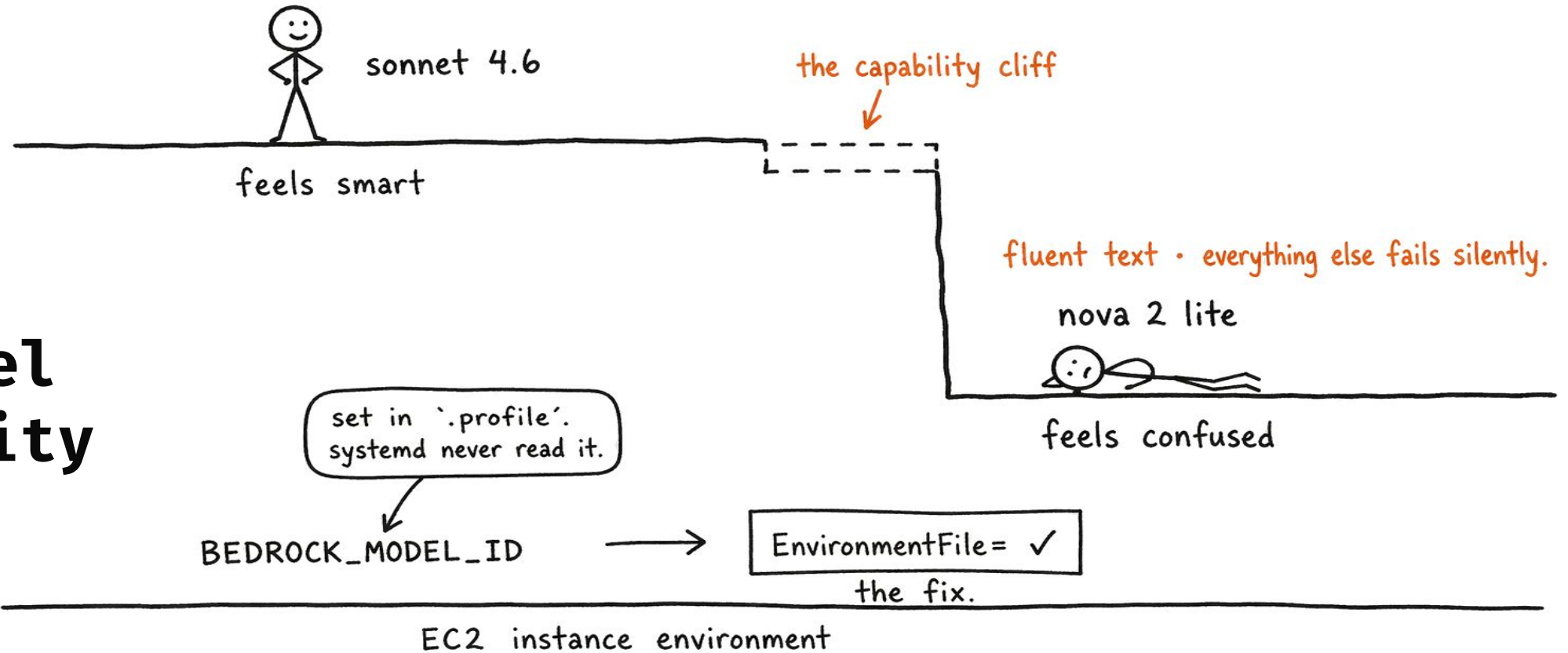


after: 1 wake-up/day • ~\$0.15/day • ~\$5-15/month

the bug was in the seam between two reasonable defaults. so was the fix - kill the heartbeat

> not all models are interchangeable

tool use ✓ • instruction-following ✓ • context handling ✓



02 the model capability cliff

when an agent feels dumb, check the env before the prompt.

> platform revelations



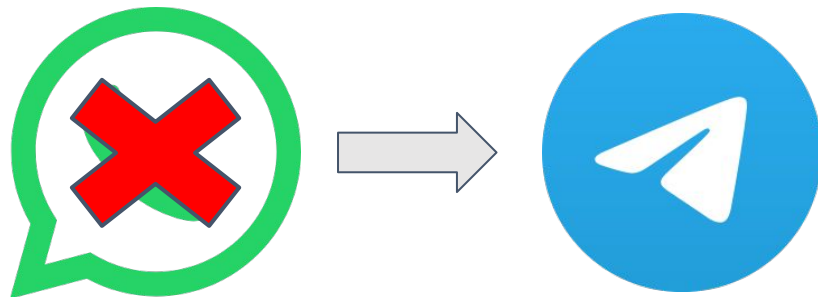
03 platform bans

Monday AM: telegram banned from previous owner. filed report.

Monday PM: whatsapp paired.

Tuesday AM: whatsapp banned.

Tuesday PM: telegram opened up.



you don't own the interface – the platform does.

infrastructure (locked down)

no public ports ✓

SSM only ✓

IAM role for AWS ✓

VPC endpoints ✓

CloudTrail ✓



the agent's tools

git push → github (anywhere)

web_fetch → any URL

shell exec → anything the OS can do

create-github-issue.sh → github API

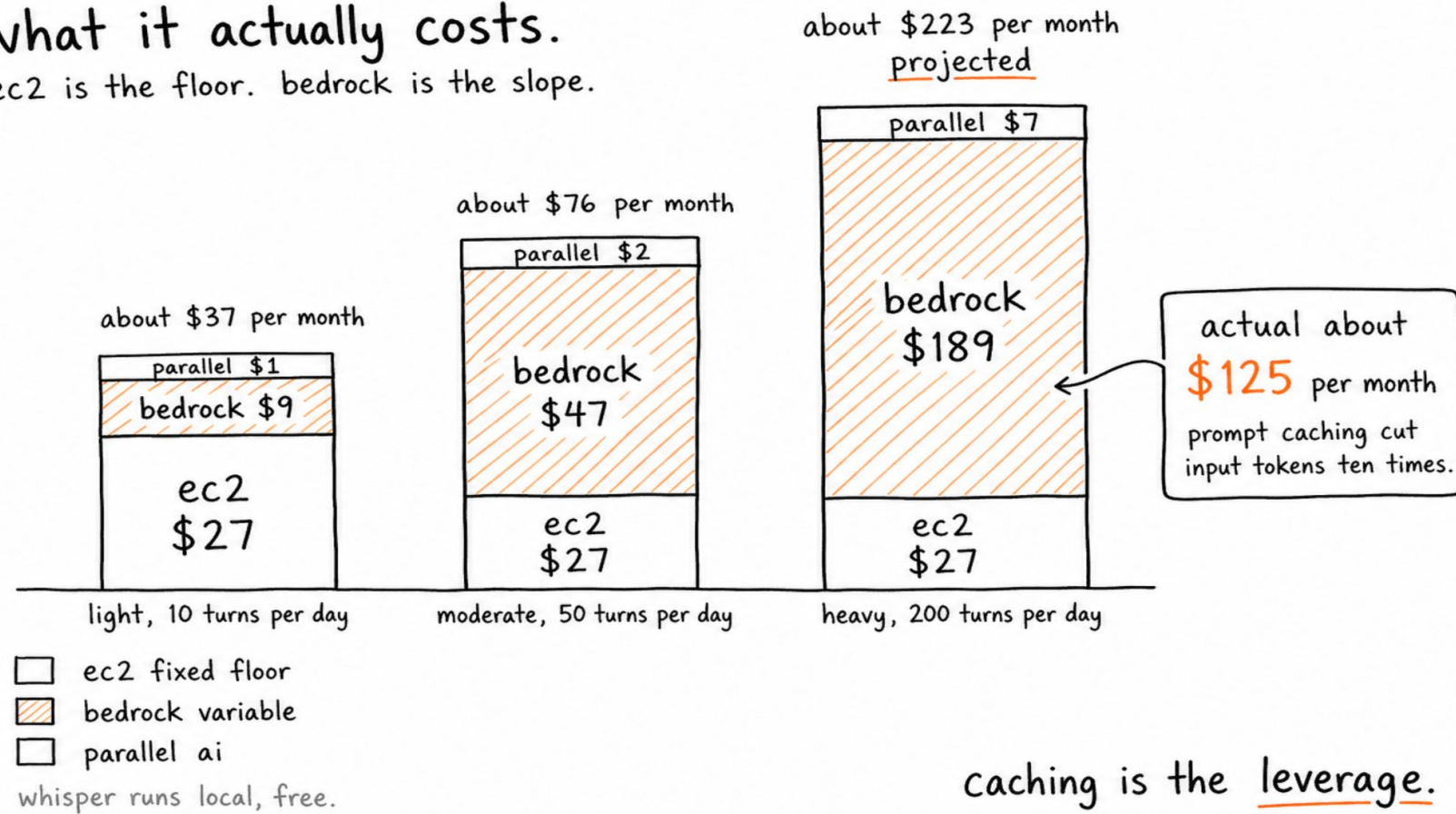
1. Secrets Manager + on-demand fetch
2. tag multimodal as untrusted
3. NemoClaw sandbox
4. IMDSv2 + egress allow-list

> node processes an EC2 instance, running AI agents who have tasks to do and a list of API's to do them, plus secret tokens.

> cheap to run, expensive enough to care about

what it actually costs.

ec2 is the floor. bedrock is the slope.



caching is the leverage.

cost isn't fixed – it's shaped by how you design the system.

> from memory → agency

what's next

email + calendar integration.

voice accuracy: Whisper+ + LLM post-pass.

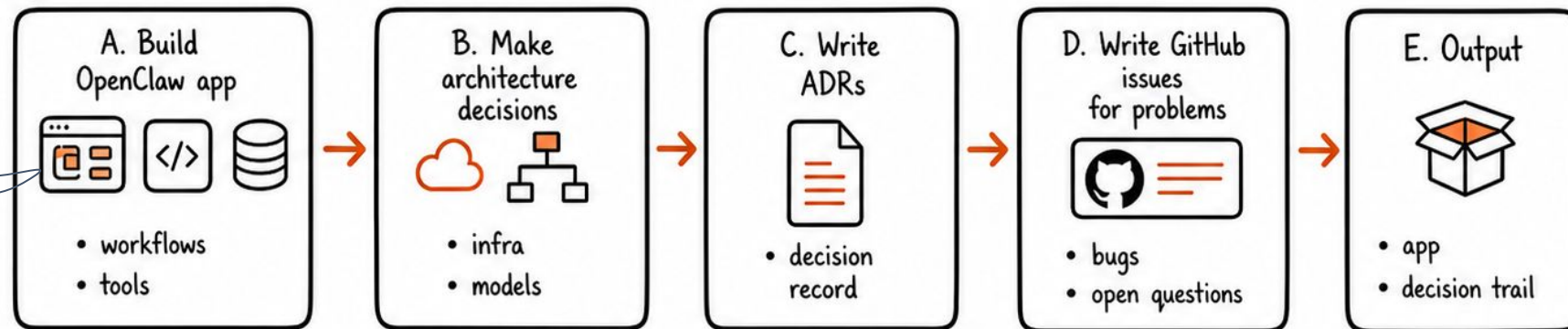
runtime isolation via subagents.

task-aware model routing

this starts as memory. it becomes agency.

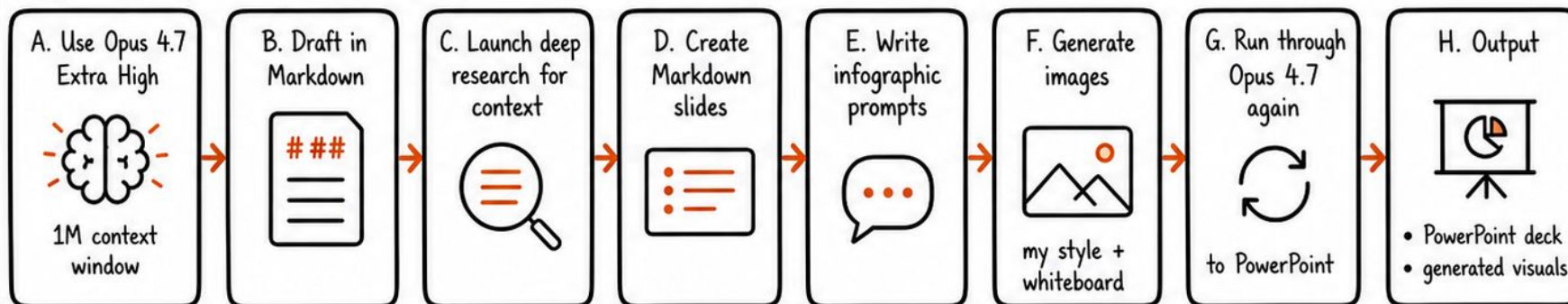
raw/
open_claw_docs/
aws_mcp/
all_research/
AWS_Sample_Repo/

① Build the app + capture architecture



Build it. Document it. Track problems.

② Use Opus 4.7 Extra High to build the presentation



Markdown bridges research, visuals, and PowerPoint.

■ ■ RVA • AI • WORKSHOP

OpenClaw Installation Party: Build Your First AI Assistant on AWS

A hands-on session with OpenClaw —
leave with a working AI assistant.



WHEN

Thu, May 14
6–9 pm



WHERE

Ippon Technologies,
Scott's Addition



TICKET

\$15 +
API credits included



SCAN TO REGISTER



> don't just watch this – try it

thank you!



01

installation party



02

code repo



03

feedback